Computer Science & IT



C Programming

Array & Pointers

Lecture No. 03



By- Abhishek Sir

Recap of Previous Lecture



- Topic: Call by reference ✓
- Topic: Function returning Pointer ✓
- Topic: Dangling references
- Topic
- Topic

auto storage

House. Destroy but address Hai

Slide

Topics to be Covered



- Topic: One Dimensional Array
- Topic
- Topic
- Topic
- Topic

Slide



Array

* Requirement is you need to maintain 100 student marks
* Declare 100 integer variable. [Management]
  * Different names
  * Different memory location } Difficult to manage

Slide



Array

A new Data type which can store multiple element of Same data type.

* Array is collection of Similar Data type..
* Array elements are stored in consecutive memory location.
  In Sequential manner.

Slide



Array

Declaration of array

int a[ ]; ← [ ] one operator

[ ] Array Index operator.

A [ ]

↓ unsigned integer

float ✗

Negative ✓

Slide

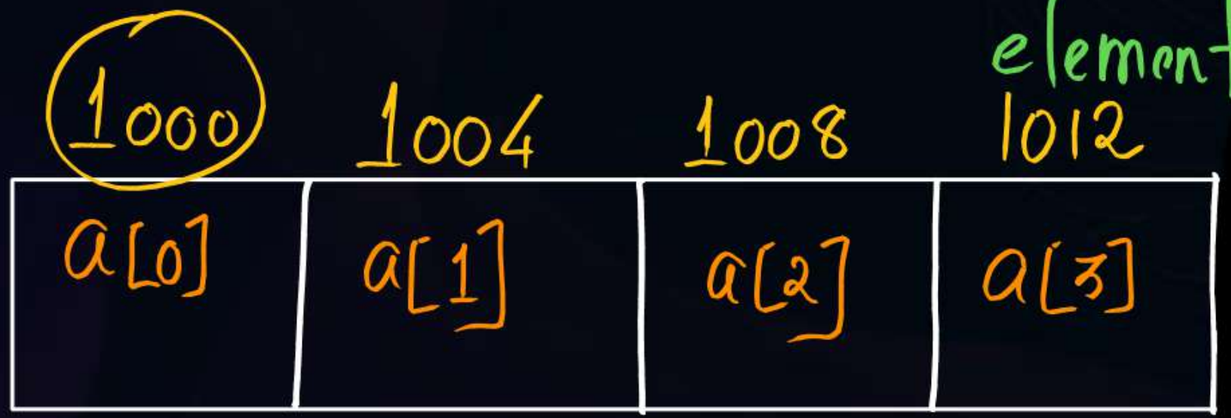
Array

A[ ] ← Index of / Subscript

↑ unsigned integer that describes No. of elements to be store.

int a[4]; ✓ a is an array of 4 integer elements

Size of element is 4 Byte



Slide



Array

Relative Addressing (COA) pointer : Indirect Addressing Mode

COA

Base Address: The starting address of array or address of first element of array

if base Address is 1000 & an integer occupy 4 Bytes

the Address of next element of array.

Slide

Byte Addressable : 1 Byte will be assign 1 Address

(8 bits)

1000 → 1 Byte

1001 - 1 Byte

1002 - 1 Byte

1003 - 1 Byte

1000 — 1003

Integer Store



Array

Index : To access element of array we Need to index value within [ ] ← operator.

First element Index is — 0

Last element index is (Size of array - 1)

int a[15]; first element a[0] Last element : a[14]

Slide



Array

Initialization of array

```
int a[4] = {1,2,3,4};
```

Index	0	1	2	3
	1	2	3	4
	1000	1004	1008	1012

value of a[0] is 1

value of a[1] is 2

value of a[2] = 3

value of a[3] = 4

to Access element 3 we Need to write a[3] → Index

Slide

Array

Initialization : `int a[] = {1,2,3,4};`

Size can be skipped if Initialization is done.

element exceeded ignore

`int a[4] = {1,2,3,4,5,6};`
`0 1 2`

Ignore

only 4 element will fit 2 Rest ignore.

`int a[4] = {11,12}`

Location blocked by default '0'

11	12	0	0

Slide



Array

Deafault storage Local array- stack
globa array : Data Segment

Slide



Array

int a[4]

1000	1004	1008	1012
11	12	13	14
a[0]	a[1]	a[2]	a[3]
21	22	23	24

int a[ ] = {11, 12, 13, 14};

```
int i;
for (i=0; i<=4; i++)
    a[i] = a[i] + 10
```

a[0]
a[1]
a[2]
a[3]
a[4] ← Array Index out of bound

```
int i = 0
for (i=0; i<=3; i++);
    a[i] = a[i] + 10;
```

a[0] = a[0] + 10
a[1] = a[1] + 10
a[2] = a[2] + 10
a[3] = a[3] + 10

Slide



Array

we observe that

if arrayIndex out of bound then its reading garbage but we try to update then

it's show "Stack Smashing detected"

ANSI C standard.



Array

```
int a[4] = {1,2,3,4};
```

1000	1004	1008	1012
1	2	3	4

```
printf("%u", a);
```

a is the Name of array

a represent address of first element of array

The first element of array is an integer

a represent address of an integer.

Slide



Array

Array Association with memory location is Not modifiable

```
int a[4] = {1,2,3,4};
```

a can't be modified or update.

Slide



Array

Pointer Arithmetic depends upon the Size of data type

int a[4] =

1	40	12	14

Addition

int *ptr;

Pointer Arithmetic

ptr = a;

ptr = ptr + 1

→ ptr = ptr + 1 x Size of data type

Slide



Array

if ptr = 1000

Suppose ptr is address of an integer then ptr+1 = 1004

ptr+1*4 = 1004

* Suppose ptr address of character

then ptr+1 = ptr+1×1 = 1000+1 = 1001

* Suppose ptr addres of float ptr+1 = ptr+1×4 = 1004

Slide



Array

ptr is a pointer

ptr = ptr + 1 = ptr + 1 x Size of Data.

Slide

Array



a[0]	a[1]	a[2]	a[3]
1	2	3	4
1000	1004	1008	1012

a is address of first element ← a

[ ]

Same Hai

$$a[i] = *(a+i) = *(i+a) = i[a]$$

$a[0] = *(a+0) = *(1000+0) = *(1000) = 1$

$a[1] = *(a+1) = *(1000+1) = *(1004) = 2$

Slide

Array

```
int a[] = {10, 49, 20, 26, 18, 40};
int *ptr;
ptr = a;
//ptr = ptr+4;
printf("%d", ptr[-1]);
```

10	49	20	26	18	40
100	104	108	112	116	120

ptr 100

ptr[-1] = *(ptr - 1) = *(116 - 1)

*(116 - 1*4) = *(112) = 26

ptr = ptr+4 = ptr + 4*4

= 100 + 4*4

= 116

26

*(100 - 1)

Slide



Array

The Rule of addition is also applied for Substraction

ptr − i = ptr − (i × Size of datatype)



2 mins Summary

Topic Array, Index

Topic Initialization

Topic Memory diagram

Topic [ ] – operator

Topic pointer Arithmetic (Imp)

Slide

THANK - YOU

